

Answers:

1.

$A \rightarrow aA \mid bbB \mid cBB$ In this grammar rule, each of the right hand side of the rule starts with a different terminal symbol. These terminal symbols are $\{a\}$ $\{b\}$ and $\{c\}$. So, this rule passes the test.

$B \rightarrow aB \mid bA \mid aBb \mid cC$ In this grammar rule, each of the right hand side of the rule does not start with a different terminal symbol. These terminal symbols are $\{a\}$ $\{b\}$ $\{a\}$ and $\{c\}$. Since both right hand side starts with $\{a\}$, this rule can not pass the test.

$C \rightarrow Aa \mid bC \mid caB$ In this grammar rule, the first right hand side starts with non terminal A, and others start with $\{b\}$ and $\{c\}$. The right hand side for the A rule starts with $\{a\}$ $\{b\}$ and $\{c\}$. Since $\{b\}$ and $\{c\}$ are repeated in the right hand sides, the rule can not pass the test.

2.

$A \rightarrow Aa \mid bbB \mid cBB$ In this rule, left recursion can be eliminated as follows:

$A \rightarrow bbBA' \mid cBBA'$

$A' \rightarrow aA' \mid \epsilon$

$B \rightarrow Ba \mid BbA \mid aBb \mid cC$ In this rule, left recursion can be eliminated as follows:

$B \rightarrow aBbB' \mid cCB'$

$B' \rightarrow aB' \mid bAB' \mid \epsilon$

$C \rightarrow CAa \mid bC \mid CcaB$ In this rule, left recursion can be eliminated as follows:

$C \rightarrow bCC'$

$C' \rightarrow AaC' \mid caBC' \mid \epsilon$

3.

For the input string **id + (id * id) \$** the shift-reduce algorithm does the below operations:

Stack	Input	Action
0	id + (id * id) \$	S5
0 id 5	+ (id * id) \$	R6
0 F 3	+ (id * id) \$	R4
0 T 2	+ (id * id) \$	R2
0 E 1	+ (id * id) \$	S6
0 E 1 + 6	(id * id) \$	S4
0 E 1 + 6 (4	id * id) \$	S5
0 E 1 + 6 (4 id 5	* id) \$	R6
0 E 1 + 6 (4 F 3	* id) \$	R4
0 E 1 + 6 (4 T 2	* id) \$	S7
0 E 1 + 6 (4 T 2 * 7	id) \$	S5
0 E 1 + 6 (4 T 2 * 7 id 5	) \$	R6
0 E 1 + 6 (4 T 2 * 7 F 10	) \$	R3
0 E 1 + 6 (4 T 2	) \$	R2
0 E 1 + 6 (4 E 8	) \$	S11

0 E 1 + 6 (4 E 8) 11	\$	R5
0 E 1 + 6 F 3	\$	R4
0 E 1 + 6 T 9	\$	R1
0 E 1	\$	accept

So, the given input string is written according to the grammar rules, and the input string is accepted.

4. The aim of the top-down and bottom-up parsers is to check whether a given input string is grammatically correct or not. To do that, both parsers generate parse tree for the given input string. The differences between them can be listed as below:

- A top-down parser generates the parse tree by starting from the root node, however a bottom-up parser generates the parse tree by starting from the leaves.
- A top-down parser can only be written for LL-grammars, however a bottom-up parser can be designed for more general grammar classes.
- A top-down parser does not need a parse table, however a bottom-up parser requires a parse table that is generated according to the grammar rules to be checked.

According to the above comparison, I prefer to use a bottom-up parser for syntax analysis, because it is more general with respect to top-down parsers, and there are many software systems that can generate parse table for a given set of grammar rules.